



Lecturer,
Dept. of CSE, ULAB, Dhaka



EMAIL:
atanu.shuvam@ulab.edu.bd

CSE 2201 : Algorithms

Lecture 09: Sorting

Atanu Shuvam Roy

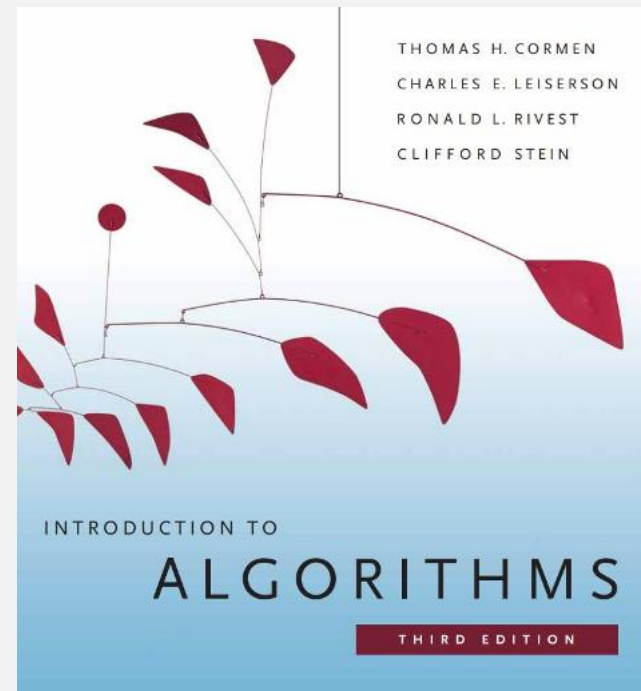


Class Code
GVWTHYJV



Reference Book

- Introduction to Algorithms, 3rd edition. T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein



Distribution of Marks

Assessments	%
Mid Term Examination	25
Final Examination	40
Attendance & class performance	10
Assignment & Quizzes	25
Total	100

Sorting problem

Ex. Student records in a university.

	Chen	3	A	991-878-4944	308 Blair
	Rohde	2	A	232-343-5555	343 Forbes
	Gazsi	4	B	766-093-9873	101 Brown
item →	Furia	1	A	766-093-9873	101 Brown
	Kanaga	3	B	898-122-9643	22 Brown
	Andrews	3	A	664-480-0023	097 Little
key →	Battle	4	C	874-088-1212	121 Whitman

Sort. Rearrange array of N items into ascending order.

Andrews	3	A	664-480-0023	097 Little
Battle	4	C	874-088-1212	121 Whitman
Chen	3	A	991-878-4944	308 Blair
Furia	1	A	766-093-9873	101 Brown
Gazsi	4	B	766-093-9873	101 Brown
Kanaga	3	B	898-122-9643	22 Brown
Rohde	2	A	232-343-5555	343 Forbes

Insertion Sort

- **Insertion sort** is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list. It is like sorting playing cards in your hands. You split the cards into two groups: the sorted cards and the unsorted cards. Then, you pick a card from the unsorted group and put it in the right place in the sorted group.

We start with the second element of the array as the first element is assumed to be sorted.

Compare the second element with the first element if the second element is smaller then swap them.

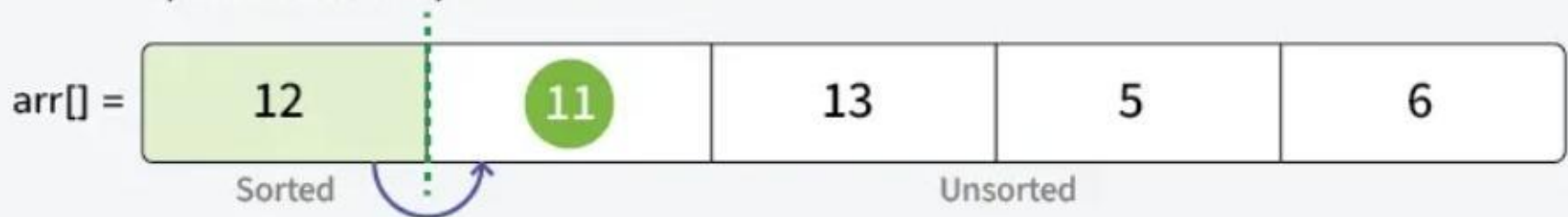
Move to the third element, compare it with the first two elements, and put it in its correct position

Repeat until the entire array is sorted.

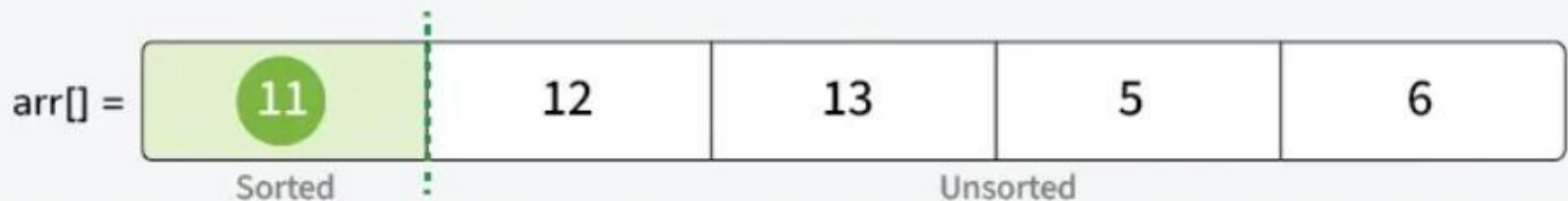
1st Pass

Key = 11

Compare key with 12. As 12 is in the wrong order, shift 12 to the right to make space for the key.



No element left to compare so insert the key at the beginning.



Insertion Sort

2nd Pass

Key = 13

Compare key with 12, 12 is in the correct order, so no changes take place.

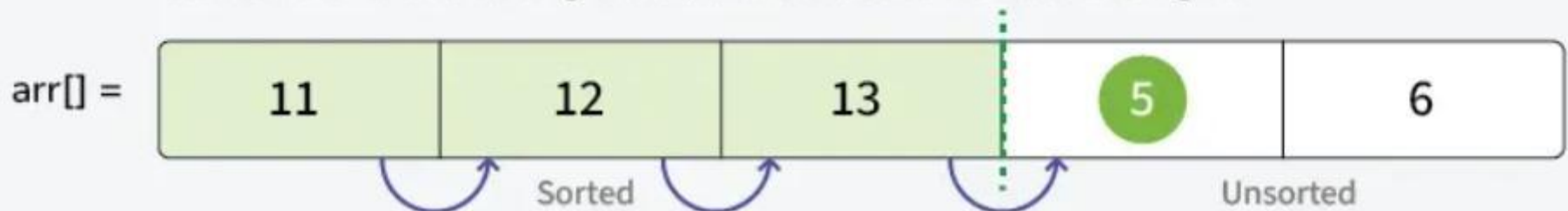


Insertion Sort

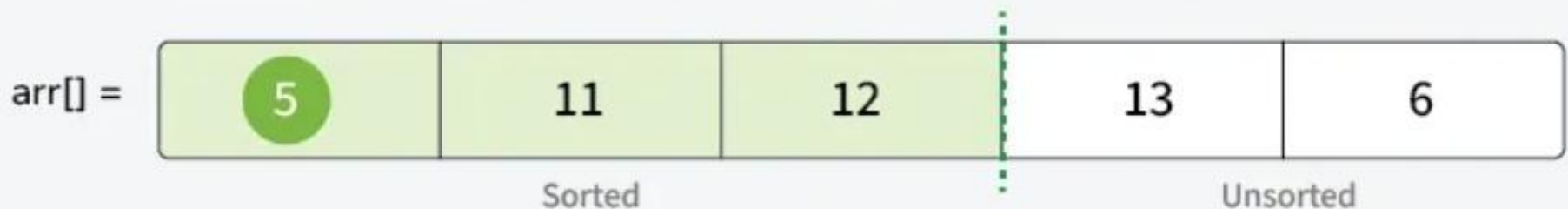
3rd Pass

Key = 5

Compare key with all the elements in the sorted subarray starting with 13, if the element is in the wrong order, shift that element to the right.



No element left to compare, so insert key at the beginning.



Insertion Sort

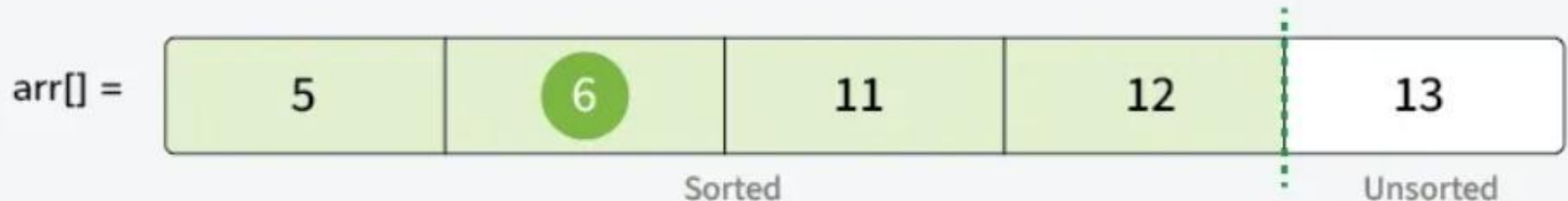
4th Pass

Key = 6

Compare key with all the elements in the sorted subarray starting with 13, if the element is in the wrong order, shift that element to the right.



5 is in the correct order, so shifting stops. Insert key after 5.



Insertion Sort

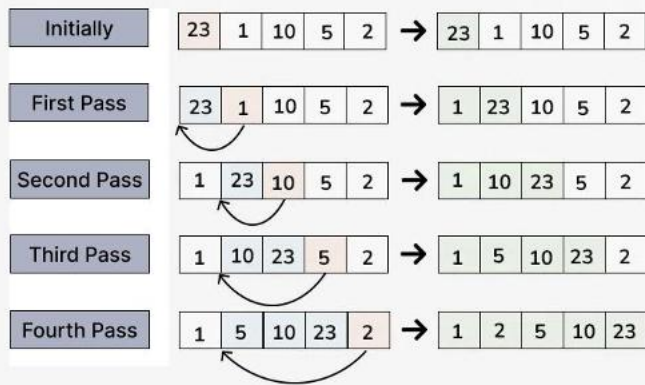
Sorted Array

The sorted part contains the whole array. Means that the whole array is sorted.

arr[] =

5	6	11	12	13
---	---	----	----	----

Sorted



Insertion Sort



arr = {23, 1, 10, 5, 2}

Initial:

- Current element is **23**
- The first element in the array is assumed to be sorted.
- The sorted part until **0th** index is : **[23]**

First Pass:

- Compare **1** with **23** (current element with the sorted part).
- Since **1** is smaller, insert **1** before **23**.
- The sorted part until **1st** index is: **[1, 23]**

Python program for implementation of Insertion Sort

Function to sort array using insertion sort

```
def insertionSort(arr):
```

```
    for i in range(1, len(arr)):
```

```
        key = arr[i]
```

```
        j = i - 1
```

```
        # Move elements of arr[0..i-1], that are
```

```
        # greater than key, to one position ahead
```

```
        # of their current position
```

```
        while j >= 0 and key < arr[j]:
```

```
            arr[j + 1] = arr[j]
```

```
            j -= 1
```

```
        arr[j + 1] = key
```

A utility function to print array of size n

```
def printArray(arr):
```

```
    for i in range(len(arr)):
```

```
        print(arr[i], end=" ")
```

```
    print()
```

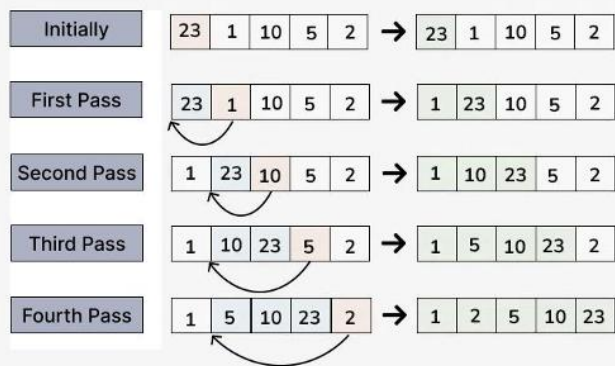
Driver method

```
if __name__ == "__main__":
```

```
    arr = [12, 11, 13, 5, 6]
```

```
    insertionSort(arr)
```

```
    printArray(arr)
```



Insertion Sort



Second Pass:

- Compare **10** with **1** and **23** (current element with the sorted part).
- Since **10** is greater than **1** and smaller than **23**, insert **10** between **1** and **23**.
- The sorted part until **2nd** index is: **[1, 10, 23]**

Third Pass:

- Compare **5** with **1**, **10**, and **23** (current element with the sorted part).
- Since **5** is greater than **1** and smaller than **10**, insert **5** between **1** and **10**.
- The sorted part until **3rd** index is: **[1, 5, 10, 23]**

Python program for implementation of Insertion Sort

Function to sort array using insertion sort

```
def insertionSort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        # Move elements of arr[0..i-1], that are
        # greater than key, to one position ahead
        # of their current position
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
```

A utility function to print array of size n

```
def printArray(arr):
    for i in range(len(arr)):
        print(arr[i], end=" ")
    print()
```

Driver method

```
if __name__ == "__main__":
    arr = [12, 11, 13, 5, 6]
    insertionSort(arr)
    printArray(arr)
```

Fourth Pass:

Compare **2** with **1**, **5**, **10**, and **23** (current element with the sorted part).

- Since **2** is greater than **1** and smaller than **5** insert **2** between **1** and **5**.
- The sorted part until **4th** index is: **[1, 2, 5, 10, 23]**

Final Array:

- The sorted array is: **[1, 2, 5, 10, 23]**

Selection Sort

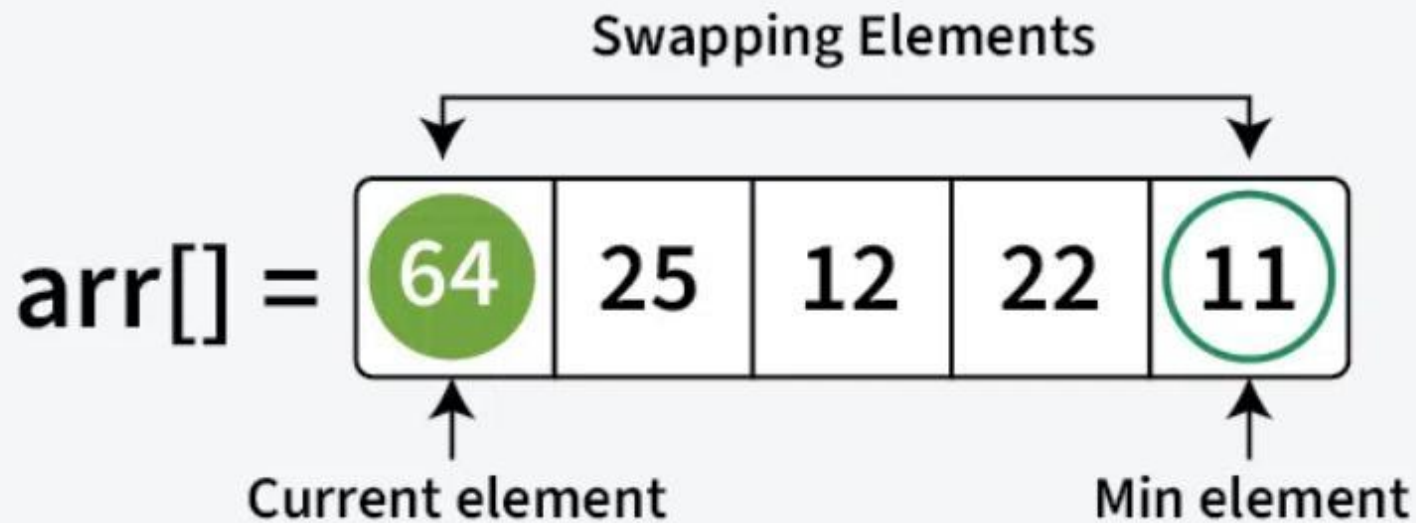
Selection Sort in 3

<https://youtu.be/RIFAkppQyGk>

01

Step

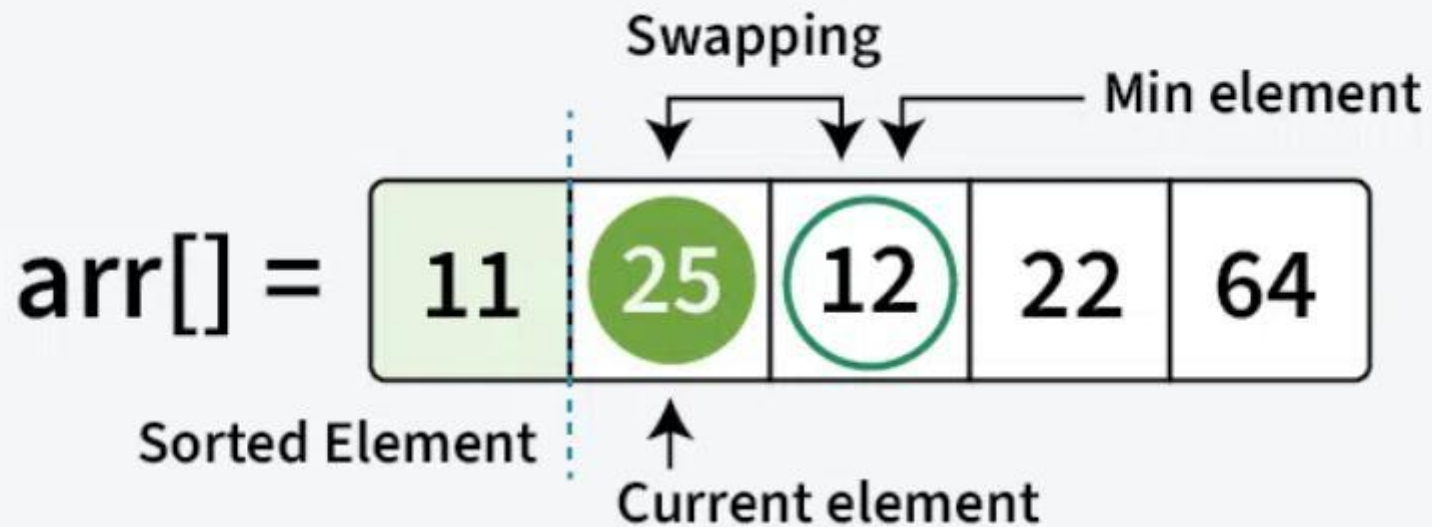
Start from the first element at index 0, find the smallest element in the rest of the array which is unsorted, and swap (11) with current element(64).



Selection Sort Algorithm

02
Step

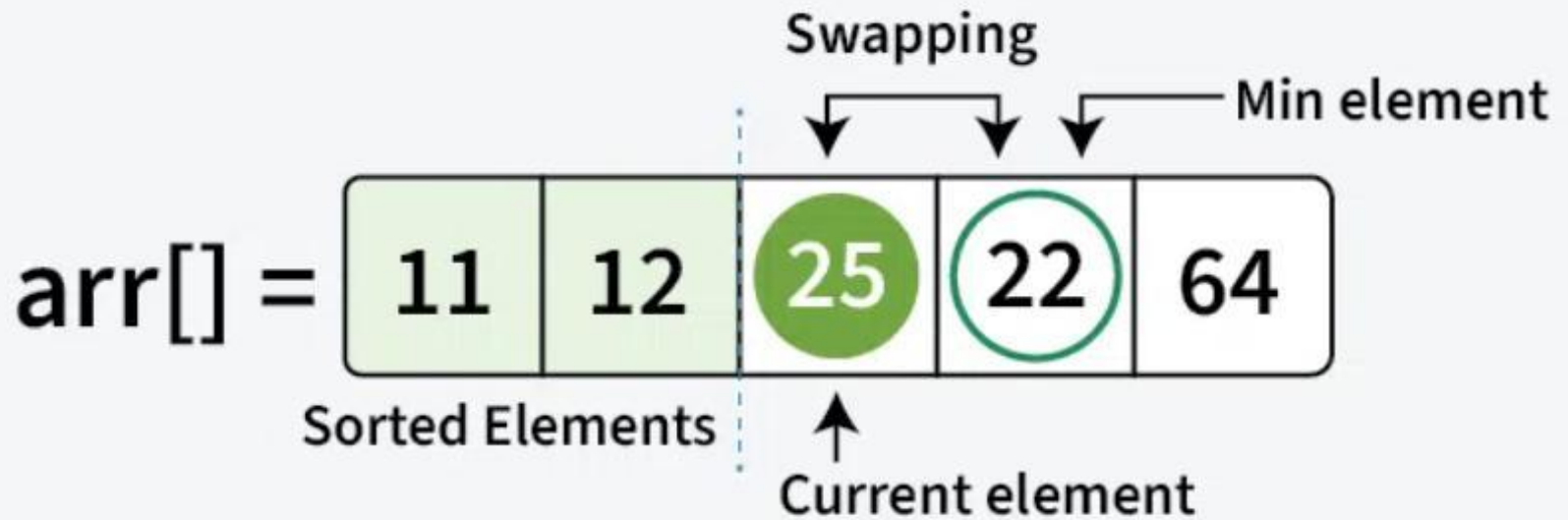
Move to the next element at index 1 (25). Find the smallest in unsorted subarray, and swap (12) with current element (25).



Selection Sort Algorithm

03
Step

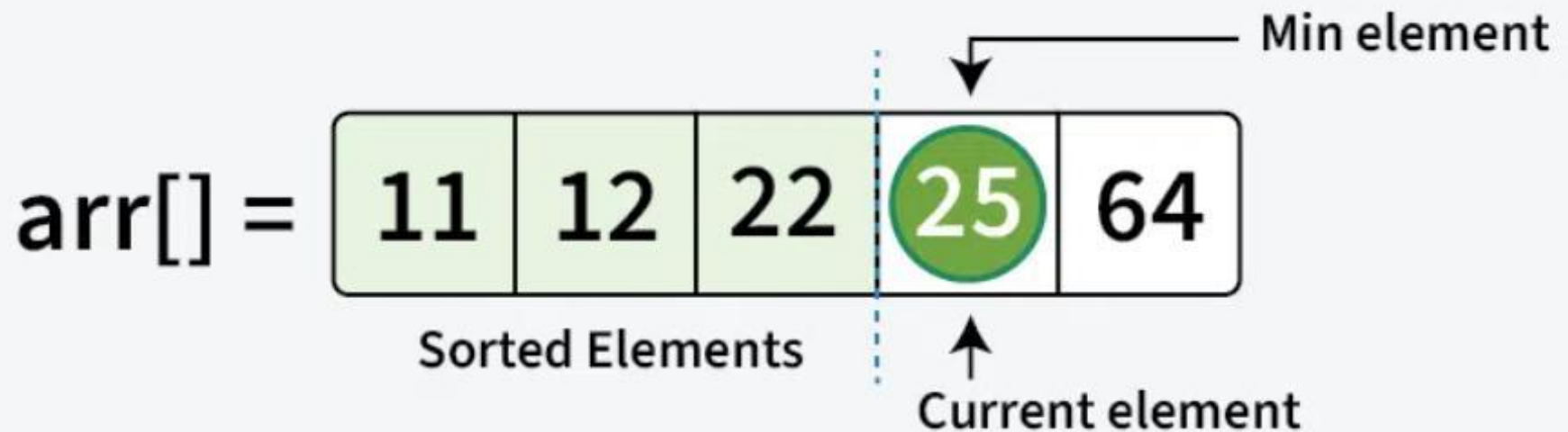
Move to element at index 2 (25). Find the minimum element from unsorted subarray, Swap (22) with current element (25).



Selection Sort Algorithm

04
Step

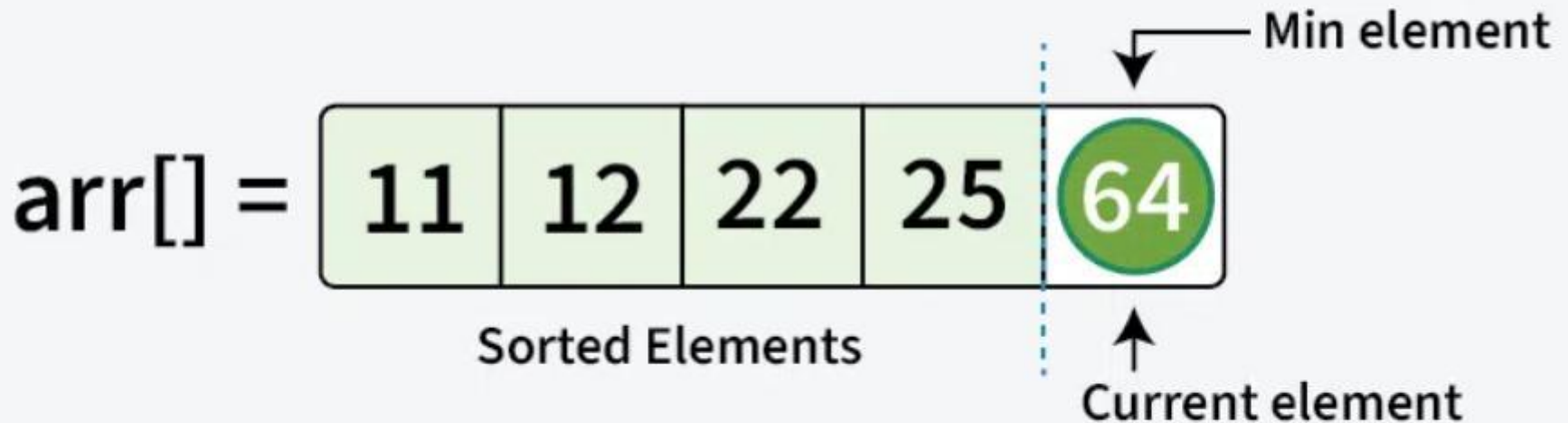
Move to element at index 3 (25), find the minimum from unsorted subarray and swap (25) with current element (25).



Selection Sort Algorithm

05
Step

Move to element at index 4 (64), find the minimum from unsorted subarray and swap (64) with current element (64).



Selection Sort Algorithm

06
Step

We get the sorted array at the end.

arr[] =

11	12	22	25	64
-----------	-----------	-----------	-----------	-----------

Sorted array

Selection Sort Algorithm

Selection Sort

- **Complexity Analysis of Selection Sort**
- **Time Complexity: $O(n^2)$** , as there are two nested loops:
- One loop to select an element of Array one by one = $O(n)$
- Another loop to compare that element with every other Array element = $O(n)$
- Therefore, overall complexity = $O(n) * O(n) = O(n*n) = O(n^2)$
- **Auxiliary Space: $O(1)$** as the only extra memory used is for temporary variables

```
def selection_sort(arr):  
    n = len(arr)  
    for i in range(n - 1):  
  
        # Assume the current position holds  
        # the minimum element  
        min_idx = i  
  
        # Iterate through the unsorted portion  
        # to find the actual minimum  
        for j in range(i + 1, n):  
            if arr[j] < arr[min_idx]:  
  
                # Update min_idx if a smaller element is found  
                min_idx = j  
  
        # Move minimum element to its  
        # correct position  
        arr[i], arr[min_idx] = arr[min_idx], arr[i]  
  
def print_array(arr):  
    for val in arr:  
        print(val, end=" ")  
    print()  
  
if __name__ == "__main__":  
    arr = [64, 25, 12, 22, 11]  
  
    print("Original array: ", end="")  
    print_array(arr)  
  
    selection_sort(arr)  
  
    print("Sorted array: ", end="")  
    print_array(arr)
```

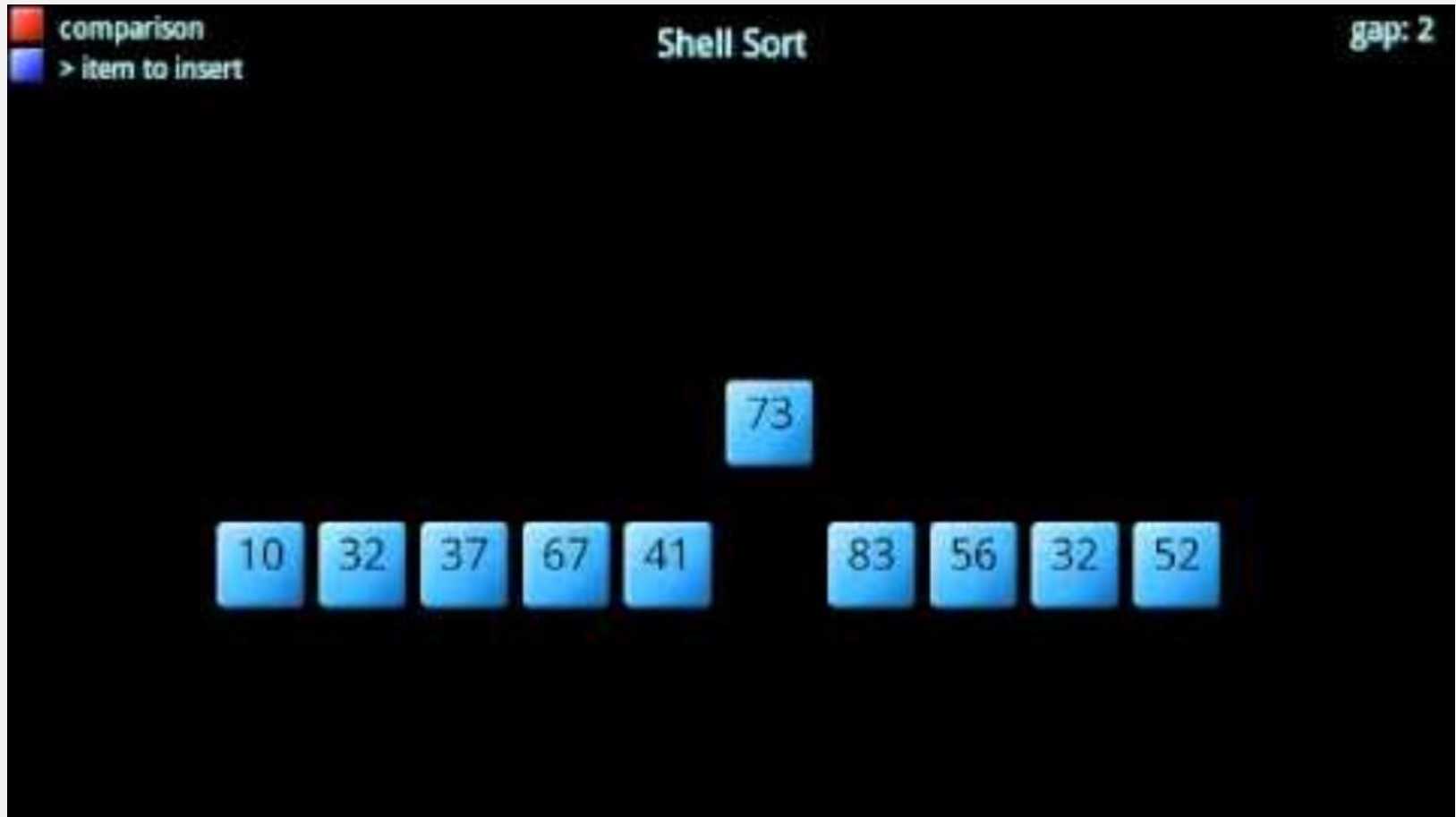
Shell Sort

- Shell Sort, also known as Shell's method, is an in-place comparison sort and an optimization of [Insertion Sort](#). It improves upon the efficiency of Insertion Sort by allowing elements to be moved over larger distances in the initial stages, which significantly reduces the number of swaps required, especially for larger datasets.
- Shell Sort was invented by Donald Shell in 1959 and is considered the first algorithm to break the $O(n^2)$ time complexity barrier for sorting.

Shell Sort

- **How Does Shell Sort Work?**
 - Shell Sort works by comparing elements that are far apart first, then gradually reducing the gap. This allows faster movement of elements across the array.
- **Here's the step-by-step explanation:**
 - Choose a gap sequence (commonly $n/2$, $n/4$, ... , 1).
 - Sort elements at each gap using Insertion Sort.
 - Reduce the gap and repeat until the gap becomes 1.

Shell Sort Animation



<https://www.youtube.com/watch?v=qzXAVXddcPU>

Shell sort

3 2 8 5 7 6 10 9



01

Initial Array & Gap Sequence

arr[] =

22	34	25	12	64	11	90	88	45
0	1	2	3	4	5	6	7	8

Gap Sequence:

Array size (n) = 9

1. First gap = $n/2 = 4$
2. Second gap = $n/4 = 2$
3. Third gap = $n/8 = 1$

02

Gap = 4

Array Update

Action

22	34	25	12	64	11	90	88	45
----	----	----	----	----	----	----	----	----

arr[0] ≤ arr[4] - Ok

22	34	25	12	64	11	90	88	45
----	----	----	----	----	----	----	----	----

arr[1] > arr[5] - Insert 11 before 34

22	11	25	12	64	34	90	88	45
----	----	----	----	----	----	----	----	----

arr[2] ≤ arr[6] - Ok

22	11	25	12	64	34	90	88	45
----	----	----	----	----	----	----	----	----

arr[3] ≤ arr[7] - Ok

22	11	25	12	64	34	90	88	45
----	----	----	----	----	----	----	----	----

arr[4] > arr[8] - Insert 45 before 64

22	11	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

Array after Gap = 4

03

Gap = 2

Array Update

Action

22	11	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

arr[0] ≤ arr[2] - Ok

22	11	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

arr[1] ≤ arr[3] - Ok

22	11	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

arr[2] ≤ arr[4] - Ok

22	11	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

arr[3] ≤ arr[5] - Ok

22	11	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

arr[4] ≤ arr[6] - Ok

22	11	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

arr[5] ≤ arr[7] - Ok

22	11	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

arr[6] > arr[8] - Insert 64 before 90

22	11	25	12	45	34	64	88	90
----	----	----	----	----	----	----	----	----

Array after Gap = 2

04

Gap = 1

Array Update

Action

22	11	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

Insert 11 before 22

11	22	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

Ok

11	22	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

Insert 12 between 11 & 22

11	12	22	25	45	34	90	88	64
----	----	----	----	----	----	----	----	----

Ok

11	12	22	25	45	34	90	88	64
----	----	----	----	----	----	----	----	----

Insert 34 between 25 & 45

11	12	22	25	34	45	90	88	64
----	----	----	----	----	----	----	----	----

Ok

11	12	22	25	34	45	90	88	64
----	----	----	----	----	----	----	----	----

Insert 88 between 45 & 90

11	12	22	25	34	45	88	90	64
----	----	----	----	----	----	----	----	----

Insert 64 between 45 & 88

11	12	22	25	34	45	64	88	90
----	----	----	----	----	----	----	----	----

Array after Gap = 1

04

Gap = 1

Array Update

Action

22	11	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

Insert 11 before 22

11	22	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

Ok

11	22	25	12	45	34	90	88	64
----	----	----	----	----	----	----	----	----

Insert 12 between 11 & 22

11	12	22	25	45	34	90	88	64
----	----	----	----	----	----	----	----	----

Ok

11	12	22	25	45	34	90	88	64
----	----	----	----	----	----	----	----	----

Insert 34 between 25 & 45

11	12	22	25	34	45	90	88	64
----	----	----	----	----	----	----	----	----

Ok

11	12	22	25	34	45	90	88	64
----	----	----	----	----	----	----	----	----

Insert 88 between 45 & 90

11	12	22	25	34	45	88	90	64
----	----	----	----	----	----	----	----	----

Insert 64 between 45 & 88

11	12	22	25	34	45	64	88	90
----	----	----	----	----	----	----	----	----

Array after Gap = 1

05 | Final Sorted Array

Since, gap size = 1. Hence, array is now completely sorted using Shell Sort:

arr[] =

11	12	22	25	34	45	64	88	90
----	----	----	----	----	----	----	----	----

Shell sort

- **Time Complexity:**
- **Best Case:** $O(n \log n)$, when the array is already nearly sorted (depends on gap sequence).
- **Average Case:** Between $O(n^{1.25})$ and $O(n^{1.5})$, depending on the chosen gap sequence.
- **Worst Case:** $O(n^2)$, with simple gap sequences like $n/2, n/4, \dots, 1$.
- **Auxiliary Space:** $O(1)$, as it sorts in-place.
Stability: Not stable (equal elements may not retain their original relative order).

```
def shell_sort(arr):
    n = len(arr)

    gap = n // 2
    while gap > 0:

        # Perform a "gapped" insertion sort for this gap size
        for i in range(gap, n):

            # Current element to be placed correctly
            temp = arr[i]
            j = i

            # Shift earlier elements that are greater than temp
            while j >= gap and arr[j - gap] > temp:
                arr[j] = arr[j - gap]
                j -= gap

            # Place temp in its correct position
            arr[j] = temp

        # Reduce the gap
        gap //= 2

# Utility function to print array
def print_array(arr):
    print(" ".join(map(str, arr)))

if __name__ == "__main__":
    arr = [12, 34, 54, 2, 3]

    shell_sort(arr)
    print_array(arr)
```

Shell sort is better than Insertion sort

- $A=[8,5,3,7,6,2,4]$
- First Insertion sort

Pass	Array	Comparisons	Swaps
1	[5, 8, 3, 7, 6, 2, 4]	1	1
2	[3, 5, 8, 7, 6, 2, 4]	2	2
3	[3, 5, 7, 8, 6, 2, 4]	3	2
4	[3, 5, 6, 7, 8, 2, 4]	4	3
5	[2, 3, 5, 6, 7, 8, 4]	5	5
6	[2, 3, 4, 5, 6, 7, 8]	6	5

Shell sort is better

=== GAP = 3 ===

Step 1: Insert 7 (from index 3)

Array Prev: [8, 5, 3, 7, 6, 2, 4]

Compare 8 (index 0) > 7 → shift

Array now after copy: [8, 5, 3, 8, 6, 2, 4]

Place 7 at index 0

Array after placement: [7, 5, 3, 8, 6, 2, 4]

Step 2: Insert 6 (from index 4)

Array Prev: [7, 5, 3, 8, 6, 2, 4]

Place 6 at index 4

Array after placement: [7, 5, 3, 8, 6, 2, 4]

Step 3: Insert 2 (from index 5)

Array Prev: [7, 5, 3, 8, 6, 2, 4]

Compare 3 (index 2) > 2 → shift

Array now after copy: [7, 5, 3, 8, 6, 3, 4]

Place 2 at index 2

Array after placement: [7, 5, 2, 8, 6, 3, 4]

Step 4: Insert 4 (from index 6)

Array Prev: [7, 5, 2, 8, 6, 3, 4]

Compare 8 (index 3) > 4 → shift

Array now after copy: [7, 5, 2, 8, 6, 3, 8]

Compare 7 (index 0) > 4 → shift

Array now after copy: [7, 5, 2, 7, 6, 3, 8]

Place 4 at index 0

Array after placement: [4, 5, 2, 7, 6, 3, 8]

=== GAP = 1 ===

Step 5: Insert 5 (from index 1)

Array Prev: [4, 5, 2, 7, 6, 3, 8]

Place 5 at index 1

Array after placement: [4, 5, 2, 7, 6, 3, 8]

Step 6: Insert 2 (from index 2)

Array Prev: [4, 5, 2, 7, 6, 3, 8]

Compare 5 (index 1) > 2 → shift

Array now after copy: [4, 5, 5, 7, 6, 3, 8]

Compare 4 (index 0) > 2 → shift

Array now after copy: [4, 4, 5, 7, 6, 3, 8]

Place 2 at index 0

Array after placement: [2, 4, 5, 7, 6, 3, 8]

Step 7: Insert 7 (from index 3)

Array Prev: [2, 4, 5, 7, 6, 3, 8]

Place 7 at index 3

Array after placement: [2, 4, 5, 7, 6, 3, 8]

Step 8: Insert 6 (from index 4)

Array Prev: [2, 4, 5, 7, 6, 3, 8]

Compare 7 (index 3) > 6 → shift

Array now after copy: [2, 4, 5, 7, 7, 3, 8]

Place 6 at index 3

Array after placement: [2, 4, 5, 6, 7, 3, 8]

Step 9: Insert 3 (from index 5)

Array Prev: [2, 4, 5, 6, 7, 3, 8]

Compare 7 (index 4) > 3 → shift

Array now after copy: [2, 4, 5, 6, 7, 7, 8]

Compare 6 (index 3) > 3 → shift

Array now after copy: [2, 4, 5, 6, 6, 7, 8]

Compare 5 (index 2) > 3 → shift

Array now after copy: [2, 4, 5, 5, 6, 7, 8]

Compare 4 (index 1) > 3 → shift

Array now after copy: [2, 4, 4, 5, 6, 7, 8]

Place 3 at index 1

Array after placement: [2, 3, 4, 5, 6, 7, 8]

Step 10: Insert 8 (from index 6)

Array Prev: [2, 3, 4, 5, 6, 7, 8]

Place 8 at index 6

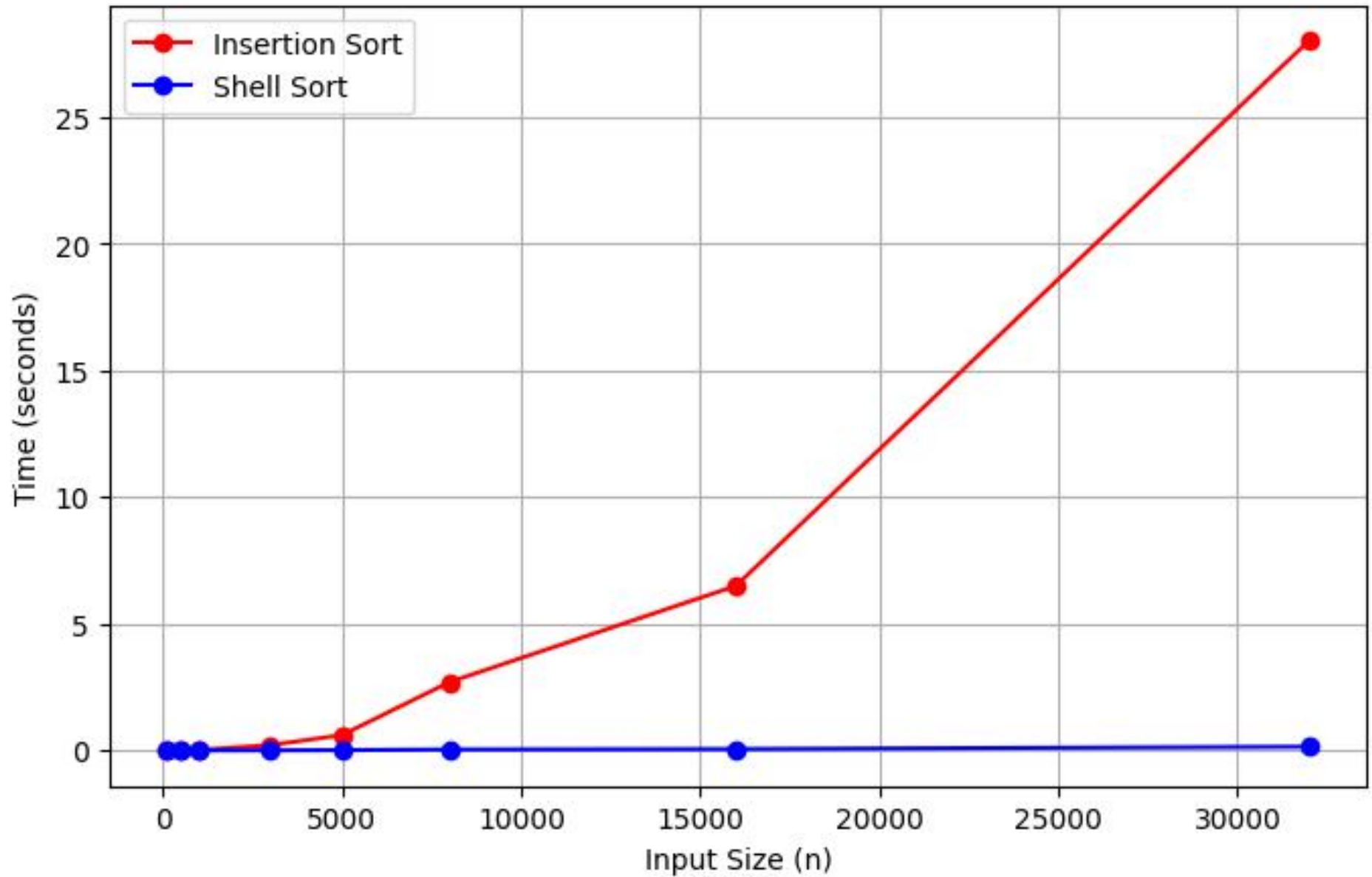
Array after placement: [2, 3, 4, 5, 6, 7, 8]

Final Sorted Array:

2 3 4 5 6 7 8

=== Code Execution Successful ===

Shell Sort vs Insertion Sort Performance



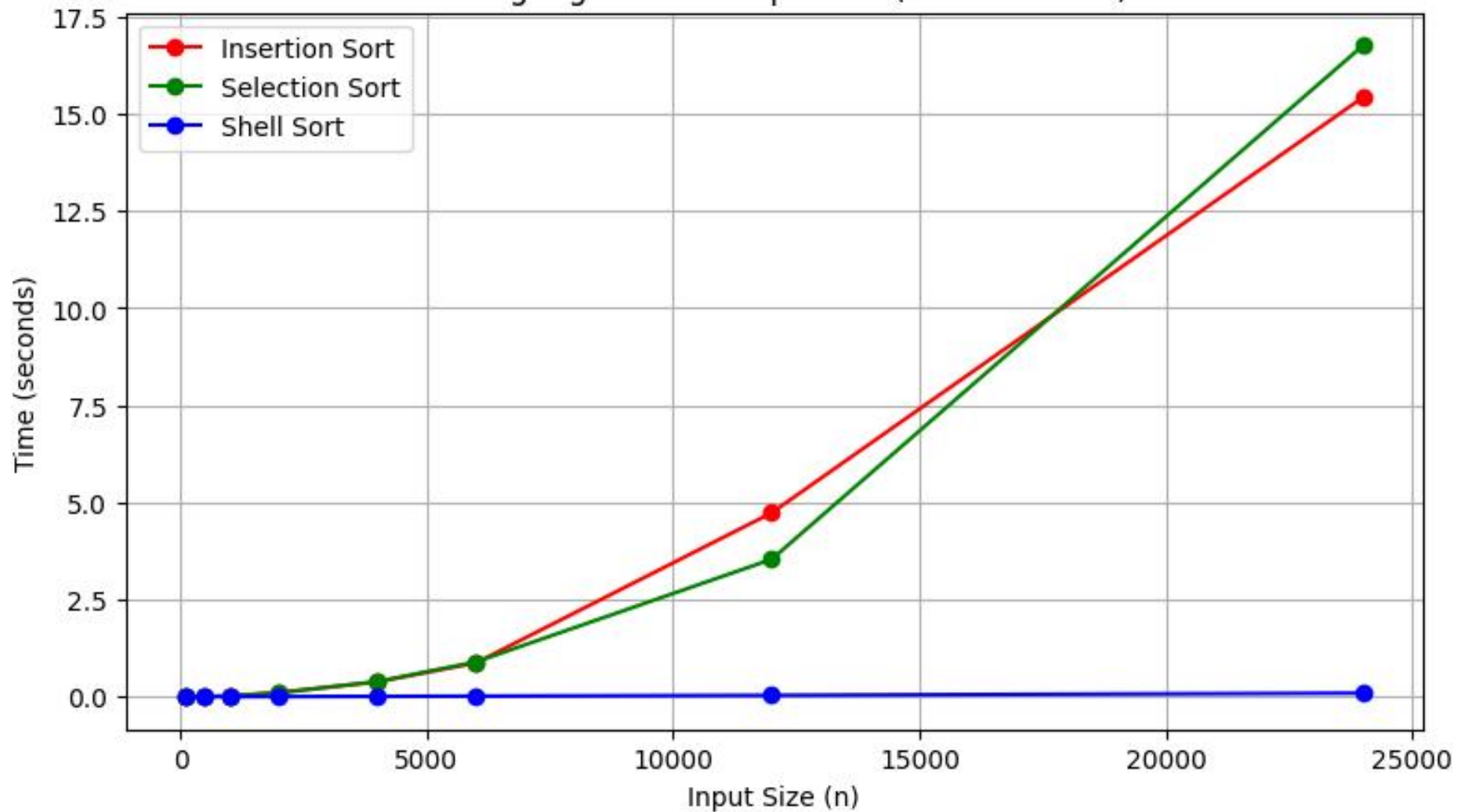
Shell sort vs Insertion Sort

Case	Insertion Sort	Shell Sort
Random data	✗ Very slow ($O(n^2)$)	✓ Much faster ($\approx O(n \log^2 n)$)
Nearly sorted data	✓ Very fast ($\approx O(n)$)	⚠ Slightly slower (unnecessary passes)
Reversed / random large input	✗ Poor	✓ Excellent

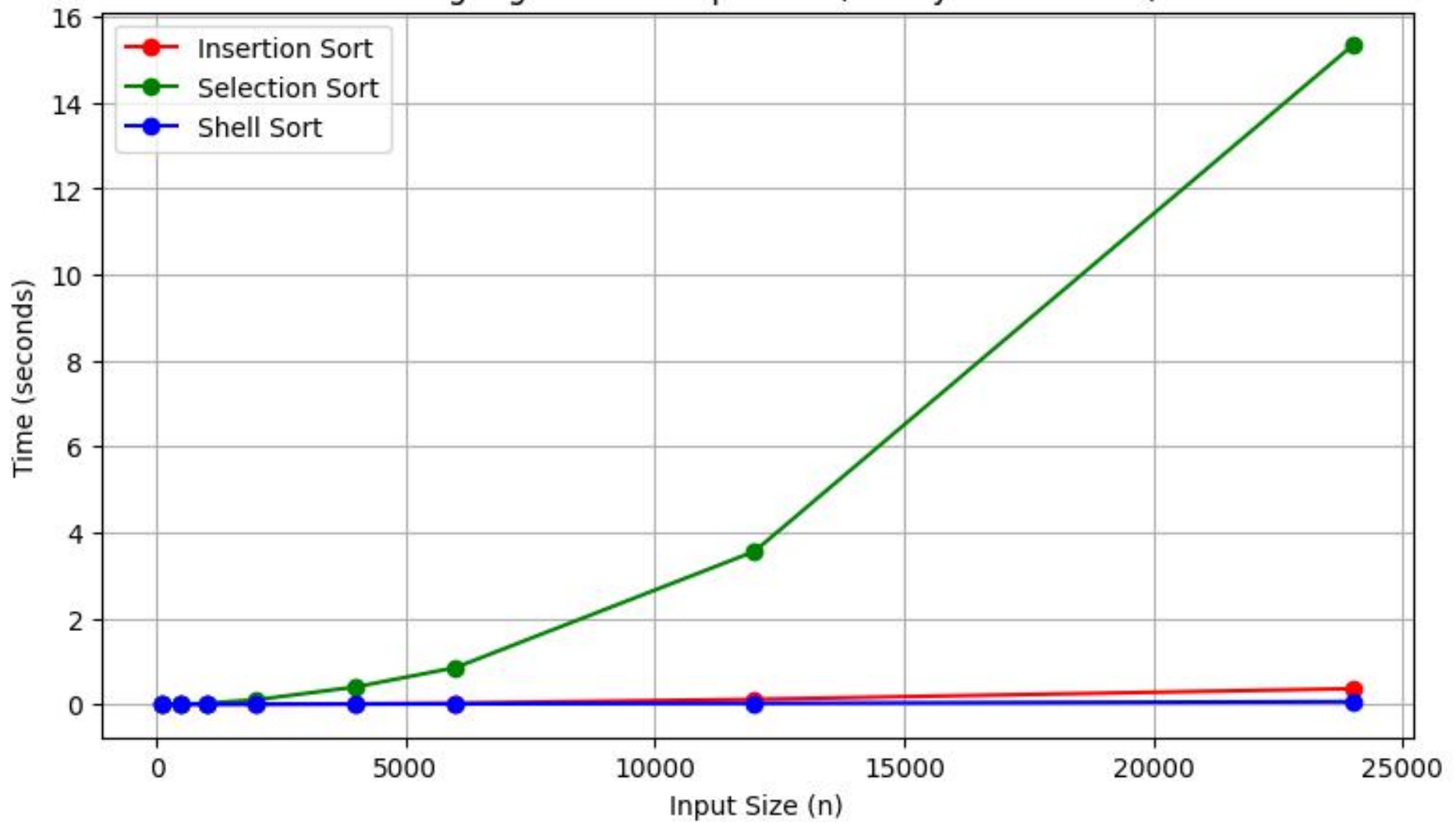
Shell vs Insertion vs Selection

Case	Insertion Sort	Selection Sort	Shell Sort
Random Data	⚠ Slow ($O(n^2)$)	⚠ Slow ($O(n^2)$)	✅ Much faster ($\approx O(n^{\{3/2\}})$)
Nearly Sorted	✅ Very fast ($\approx O(n)$)	❌ No improvement (still $O(n^2)$)	⚠ Slightly slower than Insertion
Number of Swaps	Moderate	High	Moderate
Number of Comparisons	High	High	Lower
When to Use	Small or nearly sorted data	Educational purposes only	Medium-sized random data

Sorting Algorithm Comparison (Random Data)



Sorting Algorithm Comparison (Nearly Sorted Data)



Reference

- <http://coursera.org/learn/algorithms-part1/>
- GeeksForGeeks
- YouTube
- LLMs



Thank You

atanu.Shuvam@ulab.edu.bd

University of Liberal Arts Bangladesh
Department of Computer Science & Engineering